Bevezetés a Rust programozásba

Gembela Gergely

9. óra: A kód rendszerezése és terjesztése - csomagok, crate-ek

https://github.com/bmeaut/VIAUBXAV090

Ismétlés: Crate-ek

- A kódszervezés alapeleme, ez a legkisebb fordítható egység
- Kétféle célja lehet
  - Futtatható állomány előállítása (binary)
  - Könyvtár (lib)
- A fordítás a gyökerükből indul (crate root)
- A fordításhoz szükséges adatokat a Cargo.toml fájl tartalmazza

Láthatóság az eddigieknél pontosabban

- Alapértelmezetten minden privát (ekkor az elem az azt definiáló modulon belülről érhető el)
- Csak a publikus láthatóságú elemek használhatók más modulokból
- Lehetséges
  - A crate szintjén publikussá tenni elemeket `pub(crate)`
  - A szülő modul szintjéig publikussá tenni `pub(super)`
  - Vagy szintenként szűrni `pub(in crate::mod1::modinner)`

Láthatóság az eddigieknél pontosabban

- A `self` kulcsszó hasznos lehet névütközés feloldásakor, mindig az aktuális modulra mutat
- Struktúrák mezői esetén is beszélhetünk láthatóságról
  - Ha egy mező privát, csak a saját moduljából (és annak almoduljaiból) érhető el
  - Ha publikus, bárhonnan elérhető (encapsulation)
- A `pub use ...` kifejezéssel újraexportálhatunk elemeket

Package-ek

- 1 vagy több crate gyűjteménye
  - Tetszőleges számú binary crate
  - Legfeljebb 1 library crate
- Jellemzően egy package 1 crate-et publikál
- Gyakran használják szinonímaként (pl. Crates.io-n, ahol valójában pacakage-ek vannak fent)

Cargo Workspace-ek

- Hasonló, mint a `.sln` .NET projekteknél
  - Közös kimenet könyvtár, függőségkezelés, `[patch]`, `[profile.*]`
- Tetszőleges csomag bevonható alá, egyszerre fordítható, közösen kezelhetők a függőségeik

A Crates.io

- A Rust ökoszisztéma központi „csomagközpontja”
- Nem csak könyvtárak érhetők el itt
- Hasonló a `NuGet` vagy `npm` csomagkezelőkhöz

Függőségek megadása

- Alapvetően a `crates.io`-ról adjuk meg őket, a `Cargo.toml`-ben

```
[dependencies]
time = "0.1.12"
```

- Lehetséges más registry használata
- Megadhatunk Git repo-t, vagy akár elérési utat
- Lehetséges felülbírálni függőséget pl. teszteléshez

Crate feltöltése

- Dokumentációs kommentek használata erősen ajánlott (ebből generálhatjuk a HTML dokumentációt a `cargo doc` paranccsal itt is)
- Ki kell tölteni a `[package]` metaadatokat a leíróban (legalább a verzió, egy egyedi név és egy licenc választása szükséges)
- Regisztrálni kell a `Crates.io`-n
- Végül a folyamat vége `cargo publish`

Saját registry

- Rengeteg lehetőségünk van saját registry-t hostolni
  - Akár egy megfelelően strukturált Git repo is lehet registry
- Egy lista ezekről
- Több szolgáltatás eleve tartalmaz integrálva registry-t
  - pl. Gitea

Build scriptek

- Ha létrehozunk egy `build.rs` fájlt a crate gyökerébe, írhatunk kódot, ami a build előtt lefut
  - pl. más nyelvek fordítóinak hívása, protokollgenerátorok (pl. protobuf)
  - a script alapvetően mindig lefutna, de `rerun-if` parancsokkal segíthetünk a Cargo-nak, hogy eldöntse mikor kell valóban futtatni (lehet fájlok, vagy a környezet változásához is kötni)

Profilok

- Két alapértelmezett profillal már találkoztunk (*release* és *debug*)
- Létrehozhatók saját release profilok
  - Ezek saját build beállításokat adhatnak meg

```
[profile.<profilnév>]
beállítás1 = 42
opt-level = 3
```

- A profil felhasználása `cargo build --profile name`

Features

- Feltételes fordítást valósíthatunk meg velük
- A `Cargo.toml` `[features]` táblájában adhatjuk meg őket

```
[features]
f1 = [] #[] a függőséglista a többi feature-re
f2 = ["f1"]
default = ["f1", "f2"]
```

- Alapértelmezetten mind ki vannak kapcsolva
  - A `default`-tal ezt felülbírálhatjuk
  - Kivéve, ha a `default-features = false` flaget használjuk

Features

- A kódban pl. a `#[cfg(feature = "f1")]` attribútummal kapcsolhatjuk ki vagy be elemek fordítását
- Opcionális függőségek megadására is használhatók

```
[dependencies]
gif = { version = "0.11.1", optional = true }
[features]
gif = ["dep:gif"]
```

- Függőségek esetén a Tokio használatakor már megismert módon lehet őket engedélyezni

Más nyelvek könyvtárainak linkelése

- Nyelvfüggő, hogy mennyire nehéz
- Ha egy nyelv beszél C-t, általában tudjuk használni Rust-tal
- Több nyelvhez elérhetők eszközök, amik jelentősen csökkentik a felhasználás komplexitását
  - PyO3
  - UniFFI C#
  - Bindgen

WebAssembly target

- Egyelőre nem realisztikus, hogy a teljes webes stack csak WASM alapon működjön
  - Még mindig nincs WASM DOM hozzáférés (és nem is várható)
- Kiszámítható teljesítmény
  - Nincs GC, nincs JIT
- Érdemes pl. itt a SolidJS/Svelte teljesítményét összevetni

ABI kompatibilitás

- A Rust ABI **nem** stabil
  - Ez egy tudatos döntés, mivel így fejlődhet a fordító és a nyelvi elemek sincsenek ennyire alacsony szinten megkötve
- A Rust támogat más szabványos ABI hívásokat (pl. `C` vagy `stdcall`)

```
extern "C" fn callableFromC(){}
```

Bindgen

- Automatikusan generál FFI binding könyvtárat C/C++ könyvtárakhoz
  - Egy wrapper segítségével akár safe kódból hívhatóvá is tehetjük
- Általában a `build.rs`-ből használjuk könyvtárként

Saját Cargo parancsok

- Ha a rendszer `PATH` változójában a cargo `cargo-valami` formátumú exe-t talál, az cargo parancsként (is) futtathatóvá válik
- A `cargo --list` paranccsal listázható minden rendelkezésre álló parancs
- A `cargo install` így képes saját magát kiegészítő eszközöket is telepíteni

Köszönöm a figyelmet!